

Multimodal Large Model + Color Blocks Back In Place

Function Description

After the program runs, first input the command to remember the current color block positions in the terminal. After the program completes recording, shuffle the color block positions. Then input the command to return the color blocks on the desktop to their original positions in the terminal. After thinking, the large model will plan an action list, and the program will control the robotic arm to complete the color block reset task based on the action functions in the action list.

Startup

Open the terminal and enter the following command:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=True
```

Then open a second terminal and enter the following command:

```
ros2 run text_chat text_chat
```

Now input the command to remember the current color block positions in the terminal. You can refer to the following examples:

```
Remember the current positions of the color blocks
```

```
root@yahboom:~# ros2 run text_chat text_chat
user input: Remember the current positions of the color blocks
okay@, let me think for a moment... [[INFO] [1775127073.152933423] [text_chat_node]: Decision making AI planning:1. Call 'seewhat()' to observe the environment;
2. Call 'seewhat()' again to observe the environment, then determine which color block is the top block;
3. If there are two or more piles of color blocks in the environment, select and execute the following content based on whether a color block is a single one. Calculate for each pile of color blocks. Only one action function can be output at a time here:
(1) If it is a single color block, pass the outer frame coordinates of the color block into the function 'compute_pose(x1, y1, x2, y2, 'name')', where 'name' represents the color of the color block, and its value can be 'red', 'blue', 'green', or 'yellow';
(2) If it is not a single color block, pass the outer frame coordinates of the top color block into the function 'compute_pose_order(x1, y1, x2, y2, 'name', 'order')', where 'name' represents the color of the color block (with values 'red', 'blue', 'green', or 'yellow'), and 'order' represents the top-to-bottom arrangement order of the color blocks. Its value is a string composed of the initial letters of the colors of each layer of color blocks from top to bottom. For example, if the top-to-bottom arrangement order of the color blocks is red, blue, green, yellow, then the value of 'order' is 'rbgy', and so on.
4. If there is only one pile of color blocks, pass the outer frame coordinates of the top color block into the function 'compute_pose_order(x1, y1, x2, y2, 'name', 'order')', where 'name' represents the color of the color block (with values 'red', 'blue', 'green', or 'yellow'), and 'order' represents the top-to-bottom arrangement order of the color blocks. Its value is a string composed of the initial letters of the colors of each layer of color blocks from top to bottom. For example, if the top-to-bottom arrangement order of the color blocks is red, blue, green, yellow, then the value of 'order' is 'rbgy', and so on.
[INFO] [1775127076.090205402] [text_chat_node]: "action": ['seewhat()'], "response": "Got it! I'm going to remember where all the color blocks are. Let me take a look first!"
[INFO] [1775127082.756586748] [text_chat_node]: "action": ['seewhat()'], "response": "I see three color blocks: yellow, red, and green. Let me take a closer look to figure out their positions!"
[INFO] [1775127090.516538377] [text_chat_node]: "action": ["compute_pose(0, 100, 150, 350, 'yellow')"], "response": "I see the yellow block being moved, but I still need to confirm the positions of all the blocks. Let me record their current locations!"
[INFO] [1775127101.534464855] [text_chat_node]: "action": ["compute_pose(200, 0, 410, 160, 'red')"], "response": "I've recorded the position of the yellow block! Now, let me check the red block's location."
[INFO] [1775127111.418947903] [text_chat_node]: "action": ["compute_pose(250, 180, 480, 430, 'green')"], "response": "The red block's position is saved! Now, I'll record the green block's location too."
[INFO] [1775127122.411888245] [text_chat_node]: "action": ['finishtask()'], "response": "All the color blocks' positions have been recorded successfully! I'm ready for the next task."
█
```

After recording the color block positions, input the end task command in the terminal:

```
End the position recording task
```

Then you can shuffle the color block positions. After shuffling, input the color block reset command in the terminal. You can refer to:

```
Return the blocks on the desktop to their original positions
```

```

user input: Return the blocks on the desktop to their original positions
okay@, let me think for a moment... /[INFO] [1775127293.138789488] [text_chat_node]: Decision making AI planning:1. First, call the 'color_back_to_orin()' function to control the color blocks to return to their original positions.
2. Call the 'check_remove(color)' function to check whether the target color block exists in the removal list (where 'color' is the color of the target color block, and its value range is 'red', 'blue', 'green', or 'yellow').
3. If the target color block exists in the removal list, perform the following operations:
  (1) Call the 'grasp_from_remove_list(color)' function to grasp the target color block from the removal list (the meaning of the 'color' parameter is the same as above);
  (2) Pass the color of the target color block as a parameter into the 'return_to_orin(color)' function. For example, when the target color block is red, call 'return_to_orin(red)' (the value range of the 'color' parameter is the same as above).
4. If the target color block does not exist in the removal list, follow the steps below to put the color block back to its original position (only one action function can be output at a time):
  (1) Call the 'seewhat()' function to observe the environment;
  (2) Determine whether there are other color blocks on top of the target color block that needs to be returned to its original position;
  (3) If there are no other color blocks on top of the target color block that needs to be returned to its original position, perform the following steps:
    ◉ Call the 'grasp_obj(x1, y1, x2, y2)' function to grasp the target color block that needs to be returned to its original position (where '(x1, y1, x2, y2)' are the border coordinates of the top surface of the target color block that needs to be returned to its original position);
    ◉ Pass the color of the target color block as a parameter into the 'return_to_orin(color)' function (the value range of the 'color' parameter is the same as above).
  (4) If there are other color blocks on top of the target color block that needs to be returned to its original position, perform the following steps:
    ◉ Call the 'remove_obj(x1, y1, x2, y2, color)' function to remove the color block (where '(x1, y1, x2, y2)' are the border coordinates of the top surface of this color block, and 'color' is the color of this color block, with a value range of 'red', 'blue', 'green', or 'yellow');
    ◉ Call the 'seewhat()' function to observe the environment;
    ◉ If there are no other color blocks above the target color block that needs to be returned to its original position, call the 'grasp_obj(x1, y1, x2, y2)' function to grasp the target color block (where '(x1, y1, x2, y2)' are the border coordinates of the top surface of the target color block);
    ◉ Pass the color of the target color block as a parameter into the 'return_to_orin(color)' function (the value range of the 'color' parameter is the same as above).
[INFO] [1775127295.637091377] [text_chat_node]: "action": ['color_back_to_orin()'], "response": "I'm going to return all the blocks on the desktop to their original positions. Let me start by checking if there are any blocks in the removal list.
[INFO] [1775127298.559595731] [text_chat_node]: "action": ['check_remove('green')'], "response": "I need to check if the green block is in the removal list before returning it to its original position.
[INFO] [1775127301.237892451] [text_chat_node]: "action": ['seewhat()'], "response": "The green block is not in the removal list, so I'll need to check if there are any other blocks on top of it before moving it.
[INFO] [1775127308.936447782] [text_chat_node]: "action": ['grasp_obj(50, 60, 210, 220)'], "response": "I can see the green block is not covered by any other blocks, so I'll go ahead and grasp it to return it to its original position.
[INFO] [1775127336.118697487] [text_chat_node]: "action": ['return_to_orin('green')'], "response": "I've grasped the green block. Now, I'll return it to its original position.
[INFO] [1775127346.934183907] [text_chat_node]: "action": ['check_remove('red')'], "response": "I need to check if the red block is in the removal list before returning it to its original position.
[INFO] [1775127350.327650876] [text_chat_node]: "action": ['seewhat()'], "response": "The red block is not in the removal list, so I'll check if there are any other blocks on top of it before moving it.
[INFO] [1775127358.740589262] [text_chat_node]: "action": ['grasp_obj(50, 60, 210, 220)'], "response": "I can see the red block is not covered by any other blocks, so I'll go ahead and grasp it to return it to its original position.
[INFO] [1775127386.690532688] [text_chat_node]: "action": ['return_to_orin('red')'], "response": "I've grasped the red block. Now, I'll return it to its original position.
[INFO] [1775127398.498246013] [text_chat_node]: "action": ['check_remove('yellow')'], "response": "I need to check if the yellow block is in the removal list before returning it to its original position.
[INFO] [1775127402.594439814] [text_chat_node]: "action": ['seewhat()'], "response": "The yellow block is not in the removal list, so I'll check if there are any other blocks on top of it before moving it.
[INFO] [1775127412.528910495] [text_chat_node]: "action": ['grasp_obj(0, 0, 230, 140)'], "response": "I can see the yellow block is not covered by any other blocks, so I'll go ahead and grasp it to return it to its original position.
[INFO] [1775127441.938348908] [text_chat_node]: "action": ['return_to_orin('yellow')'], "response": "I've grasped the yellow block. Now, I'll return it to its original position.
[INFO] [1775127456.434201391] [text_chat_node]: "action": ['finishtask()'], "response": "I have successfully returned all the blocks to their original positions. Everything is back in place!

```

After the buzzer beeps once, it starts gripping the color blocks and returning them to their original positions in the order recorded by the program. If there are other color blocks on top of the color block that needs to be returned, it will first remove the color blocks on top, then grip and return to the original position.

Task Planning

Record Current Color Block Positions

1. Call `seewhat()` to observe the environment;
2. Call `seewhat()` again to observe the environment, then determine which color block is the top block;
3. If there are two or more piles of color blocks in the environment, select and execute the following content based on whether a color block is a single one. Calculate for each pile of color blocks. Only one action function can be output at a time here:
 - (1) If it is a single color block, pass the outer frame coordinates of the color block into the function `compute_pose(x1, y1, x2, y2, 'name')`, where `name` represents the color of the color block, and its value can be `red`, `blue`, `green`, or `yellow`;
 - (2) If it is not a single color block, pass the outer frame coordinates of the top color block into the function `compute_pose_order(x1, y1, x2, y2, 'name', 'order')`, where `name` represents the color of the color block (with values `red`, `blue`, `green`, or `yellow`), and `order` represents the top-to-bottom arrangement order of the color blocks. Its value is a string composed of the initial letters of the colors of each layer of color blocks from top to bottom. For example, if the top-to-bottom arrangement order of the color blocks is red, blue, green, yellow, then the value of `order` is `rbgy`, and so on.
4. If there is only one pile of color blocks, pass the outer frame coordinates of the top color block into the function `compute_pose_order(x1, y1, x2, y2, 'name', 'order')`, where `name` represents the color of the color block (with values `red`, `blue`, `green`, or `yellow`), and `order` represents the top-to-bottom arrangement order of the color blocks. Its value is a string composed of the initial letters of the colors of each layer of color blocks from top to bottom. For example, if the top-to-bottom arrangement order of the color blocks is red, blue, green, yellow, then the value of `order` is `rbgy`, and so on.

Color Blocks Back In Place

1. First, call the `color_back_to_orin()` function to control the color blocks to return to their original positions.
2. Call the `check_remove(color)` function to check whether the target color block exists in the removal list (where `color` is the color of the target color block, and its value range is `red`, `blue`, `green`, or `yellow`).
3. If the target color block exists in the removal list, perform the following operations:
 - (1) Call the `grasp_from_rm_list(color)` function to grasp the target color block from the removal list (the meaning of the `color` parameter is the same as above);
 - (2) Pass the color of the target color block as a parameter into the `return_to_orin(color)` function. For example, when the target color block is red, call `return_to_orin(red)` (the value range of the `color` parameter is the same as above).
4. If the target color block does not exist in the removal list, follow the steps below to put the color block back to its original position (only one action function can be output at a time):
 - (1) Call the `seewhat()` function to observe the environment;
 - (2) Determine whether there are other color blocks on top of the target color block that needs to be returned to its original position;
 - (3) If there are no other color blocks on top of the target color block that needs to be returned to its original position, perform the following steps:
 - ① Call the `grasp_obj(x1, y1, x2, y2)` function to grasp the target color block that needs to be returned to its original position (where `(x1, y1, x2, y2)` are the border coordinates of the top surface of the target color block that needs to be returned to its original position);
 - ② Pass the color of the target color block as a parameter into the `return_to_orin(color)` function (the value range of the `color` parameter is the same as above).
 - (4) If there are other color blocks on top of the target color block that needs to be returned to its original position, perform the following steps:
 - ① Call the `remove_obj(x1, y1, x2, y2, color)` function to remove the color block (where `(x1, y1, x2, y2)` are the border coordinates of the top surface of this color block, and `color` is the color of this color block, with a value range of `red`, `blue`, `green`, or `yellow`);
 - ② Call the `seewhat()` function to observe the environment;
 - ③ If there are no other color blocks above the target color block that needs to be returned to its original position, call the `grasp_obj(x1, y1, x2, y2)` function to grasp the target color block (where `(x1, y1, x2, y2)` are the border coordinates of the top surface of the target color block);
 - ④ Pass the color of the target color block as a parameter into the `return_to_orin(color)` function (the value range of the `color` parameter is the same as above).

Core Code Analysis

compute_pose_order Record Stacked Color Block Positions

Source code path: `LargeModel_ws/src/largemodel/largemodel/action_service.py`

```
#Parameter description
#x1,y1,x2,y2: Top surface outer frame coordinates of the top color block, x1,y1 represent
#name: Color of the top color block
```

```

#order: Represents the top-to-bottom stacking order list of color blocks. For example, if
the stacking colors from top to bottom are red and yellow, then the passed order is ry
def compute_pose_order(self, x1, y1, x2, y2, name, order):
    #Get the current color block parameter
    cur_name = name.strip("\'")
    color_order = order.strip("\'")
    #Create an intermediate list to store the top-to-bottom stacking order of color blocks
    order_list = list(color_order[1:])
    self.get_logger().info(f"order_list:{order_list}")
    self.cur_name = cur_name
    self.cur_pose[self.cur_name] = []
    #Start calculating position information
    cmd1 = "ros2 run largemodel_arm Record_pose"
    proc = subprocess.Popen(
        [
            "gnome-terminal",
            "--title=compute_pose",
            "--",
            "bash",
            "-c",
            f"{cmd1}; exec bash",
        ]
    )
    self.pid_num = proc.pid
    self.get_logger().info(f"self.pid_num:{self.pid_num}")
    time.sleep(5.0)
    x1 = int(x1)
    y1 = int(y1)
    x2 = int(x2)
    y2 = int(y2)
    self.object_position_pub.publish(Int16MultiArray(data=[x1, y1, x2, y2]))

    while not self.compute_pose_future.done():
        if self.interrupt_flag:
            self.check_close_compute_pose()
            Arm.Arm_serial_servo_write6(90, 150, 12, 20, 90, 30, 1000)
            self.stop()
            return
        time.sleep(0.1)

    result = self.compute_pose_future.result()
    self.check_close_compute_pose()
    if not self.interrupt_flag:
        if result.data == "compute_pose_done":
            for i in range(0, len(order_list)):
                #Here we consider that the xy direction is consistent with the top color
                block, only the height changes, so we only need to change the z value, decreasing by about
                2.8 centimeters, which is approximately the height of one color block
                x = self.cur_pose[cur_name][0]
                y = self.cur_pose[cur_name][1]
                z = self.cur_pose[cur_name][2] - (i+1)*0.028
                if order_list[i] == 'r' or order_list[i] == 'R':
                    self.cur_pose['red'] = []

```

```

        self.cur_pose['red'].append(x)
        self.cur_pose['red'].append(y)
        self.cur_pose['red'].append(z)
    elif order_list[i] == 'b' or order_list[i] == 'B':
        self.cur_pose['blue'] = []
        self.cur_pose['blue'].append(x)
        self.cur_pose['blue'].append(y)
        self.cur_pose['blue'].append(z)
    elif order_list[i] == 'y' or order_list[i] == 'Y':
        self.cur_pose['yellow'] = []
        self.cur_pose['yellow'].append(x)
        self.cur_pose['yellow'].append(y)
        self.cur_pose['yellow'].append(z)
    elif order_list[i] == 'g' or order_list[i] == 'G':
        self.cur_pose['green'] = []
        self.cur_pose['green'].append(x)
        self.cur_pose['green'].append(y)
        self.cur_pose['green'].append(z)

    self.get_logger().info(f"self.cur_pose:{self.cur_pose}")
    #Feedback to the large model that recording the stacked color block positions is
complete
    self.action_status_pub("compute_pose_order_done", x1=x1, y1=y1, x2=x2,
y2=y2,name=name,order=order)
    else:
        self.action_status_pub("compute_pose_failed", x1=x1, y1=y1, x2=x2, y2=y2)

```

color_back_to_orin Publish Color Block Reset Sequence

Source code path: `LargeModel_ws/src/largemodel/largemodel/action_service.py`

```

def color_back_to_orin(self):
    self.back_list = True
    #Buzzer beeps once
    Arm.Arm_Buzzer_On()
    time.sleep(0.5)
    Arm.Arm_Buzzer_Off()
    #Traverse the reversed list of stored color block positions and feedback the current
color block reset command to the large model
    for key in reversed(self.cur_pose.keys()):
        if key == 'red':
            self.action_status_pub("send_red_back_to_orin")
        elif key == 'green':
            self.action_status_pub("send_green_back_to_orin")
        elif key == 'blue':
            self.action_status_pub("send_blue_back_to_orin")
        elif key == 'yellow':
            self.action_status_pub("send_yellow_back_to_orin")
    while not self.return_done:
        time.sleep(0.1)
    self.return_done = False
    self.action_status_pub("return_to_orin_done")
    self.back_list = False

```

```

"send_red_back_to_orin": "Robot feedback: Place the red block back to its original position",
"send_green_back_to_orin": "Robot feedback: Place the green block back to its original position",
"send_blue_back_to_orin": "Robot feedback: Place the blue block back to its original position",
"send_yellow_back_to_orin": "Robot feedback: Place the yellow block back to its original position"

```

check_remove(color) Check if Current Color Block to Reset Exists in Removal List

Source code path: `LargeModel_ws/src/largemodel/largemodel/action_service.py`

```

def check_remove(self,color):
    #Get the current color block color
    check_color = color.strip("\'")
    #If the current color block color can be found in the removal list, then feedback to the
    large model that the current color block can be found in the removal list
    if check_color in self.cur_rm_pose:
        self.action_status_pub("color is in the rm_list")
    else:
        self.action_status_pub("color is not in the rm_list")

"color is in the rm_list" : "The color block exists in the removal list, grip the color
block from the removal list",
"color is not in the rm_list" : "The color block does not exist in the removal list,
determine if there are other objects on top of the color block"

```

grasp_from_rm_list Grip Color Block from Removal List

Source code path: `LargeModel_ws/src/largemodel/largemodel/action_service.py`

```

def grasp_from_rm_list(self,color):
    #Get the current color block color
    tar_color = color.strip("\'")
    #According to the current color, find the corresponding robotic arm pose value in the
    removal dictionary
    tar_joints = self.cur_rm_pose.get(tar_color)
    Arm.Arm_serial_servo_write6(tar_joints[0], tar_joints[1], tar_joints[2], tar_joints[3],
tar_joints[4], tar_joints[5],2000)
    time.sleep(2.0)
    Arm.Arm_serial_servo_write(6, 140, 1000)
    time.sleep(2.0)
    Arm.Arm_serial_servo_write6(90,120,10,10,90,140,2000)
    time.sleep(2.0)
    #Feedback to the large model that gripping the color block from the removal list is
    complete
    self.action_status_pub("grasp_from_rm_list_done")

```